RL Assignment 2025-2026

Submission Report

Student ID: r1035493

Generated at: 2026-05-27 10:40:55

1. File Integrity Check

OK	tests/test1.py
OK	tests/test2.py
OK	tests/test3.py
OK	commons.py
OK	minihack_env.py
OK	compose.yaml
OK	pytest.ini

2. Trained Weights

15/15 weight files found. Missing files will cause performance tests to fail.

OK	trained_weights/tabular/monte_carlo/empty-room.pkl
OK	trained_weights/tabular/monte_carlo/cliff-minihack.pkl
OK	trained_weights/tabular/monte_carlo/room-with-monster.pkl
OK	trained_weights/tabular/sarsa/empty-room.pkl
OK	trained_weights/tabular/sarsa/cliff-minihack.pkl
OK	trained_weights/tabular/sarsa/room-with-monster.pkl
OK	trained_weights/tabular/q_learning/empty-room.pkl
OK	trained_weights/tabular/q_learning/cliff-minihack.pkl
OK	trained_weights/tabular/q_learning/room-with-monster.pkl
OK	trained_weights/dqn/empty-room.pt
OK	trained_weights/dqn/cliff-minihack.pt
OK	trained_weights/dqn/room-with-monster.pt
OK	trained_weights/ppo/empty-room.pt
OK	trained_weights/ppo/cliff-minihack.pt
OK	trained_weights/ppo/room-with-monster.pt

3. Task File Syntax

OK	task1/dummy.py
OK	task2/tabular_agent.py
OK	task2/mc-agents.py
OK	task2/td-agents.py
OK	task3/deep_agent.py
OK	task3/dqn.py
OK	task3/ppo.py

4. Test Results Summary

53/53 tests passed (100%)

5. Full Test Output

```
============================= test session starts ==============================
collecting ... collected 53 items

tests/test1.py::test_grid_world_spaces_reset_and_render[2-2] PASSED      [  1%]
tests/test1.py::test_grid_world_spaces_reset_and_render[3-4] PASSED      [  3%]
tests/test1.py::test_grid_world_spaces_reset_and_render[5-5] PASSED      [  5%]
tests/test1.py::test_grid_world_public_dynamics_to_goal[2-2] PASSED      [  7%]
tests/test1.py::test_grid_world_public_dynamics_to_goal[3-4] PASSED      [  9%]
tests/test1.py::test_grid_world_public_dynamics_to_goal[5-5] PASSED      [ 11%]
tests/test1.py::test_grid_world_action_mapping_from_interior PASSED      [ 13%]
tests/test1.py::test_slippery_grid_world_spaces_reset_and_render[3-4-1.0] PASSED [ 15%]
tests/test1.py::test_slippery_grid_world_spaces_reset_and_render[4-4-0.8] PASSED [ 16%]
tests/test1.py::test_slippery_grid_world_reset_start_pos_option PASSED   [ 18%]
tests/test1.py::test_slippery_grid_world_p1_matches_deterministic_dynamics PASSED [ 20%]
tests/test1.py::test_slippery_grid_world_p1_reaches_goal_like_grid_world PASSED [ 22%]
tests/test1.py::test_slippery_grid_world_same_seed_same_trajectory PASSED [ 24%]
tests/test1.py::test_slippery_grid_world_empirical_distribution_from_interior PASSED [ 26%]
tests/test1.py::test_slippery_grid_world_observation_always_in_space PASSED [ 28%]
tests/test1.py::test_grid_world_hardcoded_agent_policy_and_rollout[2-2] PASSED [ 30%]
tests/test1.py::test_grid_world_hardcoded_agent_policy_and_rollout[3-4] PASSED [ 32%]
tests/test1.py::test_grid_world_hardcoded_agent_policy_and_rollout[5-5] PASSED [ 33%]
tests/test1.py::test_minihack_empty_hardcoded_agent_finishes_safely PASSED [ 35%]
tests/test1.py::test_minihack_cliff_hardcoded_agent_finishes_safely PASSED [ 37%]
tests/test1.py::test_abstract_rl_task_raw_returns_and_multiple_episodes PASSED [ 39%]
tests/test1.py::test_abstract_rl_task_respects_max_steps_per_episode PASSED [ 41%]
tests/test2.py::test_monte_carlo_backward_returns_and_incremental_average PASSED [ 43%]
tests/test2.py::test_mc_agent_on_episode_end_updates_q_table PASSED      [ 45%]
tests/test2.py::test_sarsa_update_rule PASSED                            [ 47%]
tests/test2.py::test_q_learning_update_rule PASSED                       [ 49%]
tests/test2.py::test_choose_next_action_is_greedy_when_not_learning PASSED [ 50%]
tests/test2.py::test_choose_next_action_explores_when_random_draw_is_below_epsilon PASSED [ 52%]
tests/test2.py::test_choose_next_action_exploits_when_random_draw_is_above_epsilon PASSED [ 54%]
tests/test2.py::test_linear_epsilon_decay_updates_as_expected PASSED     [ 56%]
tests/test2.py::test_mc_agent_act_buffers_transitions_and_does_not_learn_during_episode PASSED [ 58%]
tests/test2.py::test_mc_agent_on_episode_end_triggers_learn_and_clears_buffer PASSED [ 60%]
tests/test2.py::test_td_agent_act_triggers_learning_from_previous_transition PASSED [ 62%]
tests/test2.py::test_td_agent_on_episode_end_triggers_final_update PASSED [ 64%]
tests/test2.py::test_monte_carlo_improves_on_empty_room PASSED           [ 66%]
tests/test2.py::test_monte_carlo_improves_on_cliff PASSED                [ 67%]
tests/test2.py::test_monte_carlo_improves_on_room_with_monster PASSED    [ 69%]
tests/test2.py::test_sarsa_improves_on_empty_room PASSED                 [ 71%]
tests/test2.py::test_sarsa_improves_on_cliff PASSED                      [ 73%]
tests/test2.py::test_sarsa_improves_on_room_with_monster PASSED          [ 75%]
tests/test2.py::test_q_learning_improves_on_empty_room PASSED            [ 77%]
tests/test2.py::test_q_learning_improves_on_cliff PASSED                 [ 79%]
tests/test2.py::test_q_learning_improves_on_room_with_monster PASSED     [ 81%]
tests/test3.py::test_dqn_soft_target_update PASSED                       [ 83%]
tests/test3.py::test_dqn_loss_matches_expected_mse PASSED                [ 84%]
tests/test3.py::test_ppo_gae_recursion_affects_policy_update_as_expected PASSED [ 86%]
tests/test3.py::test_ppo_clipping_blocks_gradient_when_ratio_exceeds_clip_range PASSED [ 88%]
tests/test3.py::test_dqn_pretrained_empty_room PASSED                    [ 90%]
tests/test3.py::test_dqn_pretrained_cliff PASSED                         [ 92%]
tests/test3.py::test_dqn_pretrained_room_with_monster PASSED             [ 94%]
tests/test3.py::test_ppo_pretrained_empty_room PASSED                    [ 96%]
tests/test3.py::test_ppo_pretrained_cliff PASSED                         [ 98%]
tests/test3.py::test_ppo_pretrained_room_with_monster PASSED             [100%]

============================== warnings summary ===============================
../usr/local/lib/python3.12/site-packages/minihack/base.py:8
  /usr/local/lib/python3.12/site-packages/minihack/base.py:8: DeprecationWarning: pkg_resources is deprecated as an API....
    import pkg_resources
```

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
======================== 53 passed, 1 warning in 3.38s =========================
```

6. Student Solution Blocks

[task1/dummy.py] block 1

 # TODO: Initialize the environment state.

```
        self.agent_pos = (0, 0)
        self.goal_pos = (self.n - 1, self.m - 1)
```

[task1/dummy.py] block 2

 # TODO: Implement reset.

```
        super().reset(seed=seed)

        self.agent_pos = (0, 0)

        observation = self.agent_pos
        info = {}

        return observation, info
```

[task1/dummy.py] block 3

 # TODO: Implement the transition dynamics.

```
        direction = {
            0: (-1, 0),  # up
            1: (1, 0),   # down
            2: (0, 1),   # right
            3: (0, -1)    # left
        }

        move = direction.get(action, (0, 0))
        new_pos = (self.agent_pos[0] + move[0], self.agent_pos[1] + move[1])

        if 0 <= new_pos[0] < self.n and 0 <= new_pos[1] < self.m:
            self.agent_pos = new_pos

        observation = self.agent_pos
        reward = -1
        terminated = self.agent_pos == self.goal_pos
        truncated = False
        info = {}

        return observation, reward, terminated, truncated, info
```

[task1/dummy.py] block 4

 # TODO: Implement render.

```
        grid = ""

        for i in range(self.n):
            for j in range(self.m):
                if (i, j) == self.agent_pos:
                    grid += "A"
                elif (i, j) == self.goal_pos:
                    grid += "G"
                else:
                    grid += "."
            grid += "\n"

        return grid
```

[task1/dummy.py] block 5

 # TODO: Initialize the environment state.

```
        self.agent_pos = (0, 0)
        self.goal_pos = (self.n - 1, self.m - 1)
```

```
        self.np_random = np.random.RandomState()
```

[task1/dummy.py] block 6

 # TODO: Implement reset.

```
        if options and "start_pos" in options:
            self.agent_pos = options["start_pos"]
        else:
            self.agent_pos = (0, 0)

        self.np_random = np.random.RandomState(seed)

        observation = self.agent_pos
        info = {}

        return observation, info
```

[task1/dummy.py] block 7

 # TODO: Implement the stochastic transition dynamics.

```
        direction = {
            0: (-1, 0),  # up
            1: (1, 0),   # down
            2: (0, 1),   # right
            3: (0, -1)    # left
        }

        left_turn = {
            0: 3,  # up -> left
            1: 2,  # down -> right
            2: 0,  # right -> up
            3: 1   # left -> down
        }

        right_turn = {
            0: 2,  # up -> right
            1: 3,  # down -> left
            2: 1,  # right -> down
            3: 0   # left -> up
        }

        executed_action = self.np_random.choice([action, left_turn[action], right_turn[action]], p=[self.p_success, (1 -...
        move = direction.get(executed_action, (0, 0))

        new_pos = (self.agent_pos[0] + move[0], self.agent_pos[1] + move[1])

        if 0 <= new_pos[0] < self.n and 0 <= new_pos[1] < self.m:
            self.agent_pos = new_pos

        observation = self.agent_pos
        reward = -1
        terminated = self.agent_pos == self.goal_pos
        truncated = False
        info = {}

        return observation, reward, terminated, truncated, info
```

[task1/dummy.py] block 8

 # TODO: Implement render.

```
        grid = ""

        for i in range(self.n):
            for j in range(self.m):
                if (i, j) == self.agent_pos:
                    grid += "A"
```

```
            elif (i, j) == self.goal_pos:
                grid += "G"
            else:
                grid += "."
        grid += "\n"

    return grid
```

[task1/dummy.py] block 9

 # TODO: Implement the hardcoded policy.

```
    row, col = state
    goal_row, goal_col = self.goal_pos
    if row < goal_row:
        return 1
    elif col < goal_col:
        return 2
    else:
        return 0
```

[task1/dummy.py] block 10

 # TODO: Initialize any state needed by act().

```
    self.agent_pos = (0, 0)
```

[task1/dummy.py] block 11

 # TODO: Implement a policy that navigates to the goal.

```
    chars = state["chars"]
    self.agent_pos = np.argwhere(chars == ord('@'))[0]

    if chars[self.agent_pos[0]][self.agent_pos[1] + 1] != ord('|'):
        return me.ACTIONS.index(nethack.CompassCardinalDirection.E)
    elif chars[self.agent_pos[0] + 1][self.agent_pos[1]] != ord('-'):
        return me.ACTIONS.index(nethack.CompassCardinalDirection.S)
    else:
        return me.ACTIONS.index(nethack.CompassCardinalDirection.N)
```

[task1/dummy.py] block 12

 # TODO: Initialize any state needed by act().

```
    self.agent_pos = (0, 0)
```

[task1/dummy.py] block 13

 # TODO: Implement a policy that navigates to the goal while avoiding hazards.

```
    chars = state["chars"]
    print(f"State chars: {chars}")
    self.agent_pos = np.argwhere(chars == ord('@'))[0]

    if chars[self.agent_pos[0]][self.agent_pos[1] + 1] == ord('L'):
        return me.ACTIONS.index(nethack.CompassCardinalDirection.N)
    elif chars[self.agent_pos[0]][self.agent_pos[1] + 1] != ord('|'):
        return me.ACTIONS.index(nethack.CompassCardinalDirection.E)
    elif chars[self.agent_pos[0] + 1][self.agent_pos[1]] != ord('-'):
        return me.ACTIONS.index(nethack.CompassCardinalDirection.S)
    else:
        return me.ACTIONS.index(nethack.CompassCardinalDirection.N)
```

[task1/dummy.py] block 14

 # TODO: Implement the interaction loop.

```
    episode_returns = []
    for episode in range(n_episodes):
        state, info = self.env.reset()
        episode_return = 0
        step = 0
```

```
            terminated = False
            truncated = False

            while not terminated and not truncated:
                action = self.agent.act(state, reward=episode_return, step=step)
                next_state, reward, terminated, truncated, info = self.env.step(action)
                episode_return += reward
                state = next_state
                step += 1

                if max_steps_per_episode is not None and step >= max_steps_per_episode:
                    truncated = True

            self.agent.onEpisodeEnd(state, action, episode_return, episode)
            episode_returns.append(episode_return)

        return episode_returns
```

[task2/tabular_agent.py] block 1

```
        if random.uniform(0, 1) < self.epsilon:
            return self.action_space.sample()
        else:
            return int(np.argmax(self.q_table[state]))
```

[task2/tabular_agent.py] block 2

```
        if  episode < self.epsilon_decay_episodes:
            self.epsilon = self.epsilon_start - ((episode + 1) * ((self.epsilon_start - self.epsilon_end) / self.epsilon...
        else:
            self.epsilon = self.epsilon_end
```

[task2/mc-agents.py] block 1

```
        if self.last_action is not None and self.last_state is not None:
            self.last_episode.append((self.last_state, self.last_action, reward))

        encoded_state = self.state_encoder(state)
        self.last_state = encoded_state
        action = self.choose_next_action(encoded_state)
        self.last_action = action

        return action
```

[task2/mc-agents.py] block 2

```
        self.last_episode.append((self.last_state, self.last_action, reward))
        self.learn()
        self.last_episode.clear()
        self.last_state = None
        self.last_action = None
```

[task2/mc-agents.py] block 3

```
        G = 0
        for state, action, reward in reversed(self.last_episode):
            G = self.gamma * G + reward
            self.counts[state][action] += 1

            self.q_table[state][action] += (G - self.q_table[state][action]) / self.counts[state][action]
```

[task2/td-agents.py] block 1

```
        encoded_state = self.state_encoder(state)
        action = self.choose_next_action(encoded_state)

        if self.last_action is not None and self.last_state is not None:
            self.learn(self.last_state, self.last_action, encoded_state, action, reward)
```

```
        self.last_state = encoded_state
        self.last_action = action

        return action
```

[task2/td-agents.py] block 2

```
        encoded_state = self.state_encoder(args[0])
        action = args[1]
        reward = args[2]
        episode = args[3]

        self.learn(self.last_state, self.last_action, encoded_state, action, reward)

        self.update_epsilon(episode)

        self.last_state = None
        self.last_action = None
```

[task2/td-agents.py] block 3

```
        self.q_table[state][action] += self.alpha * (reward + self.gamma * np.max(self.q_table[next_state]) - self.q_tab...
```

[task2/td-agents.py] block 4

```
        if self.learning and random.uniform(0, 1) < self.epsilon:
            return self.action_space.sample()
        else:
            return int(np.argmax(self.q_table[state]))
```

[task2/td-agents.py] block 5

 # TODO: Implement the SARSA update rule.

```
        self.q_table[state][action] += self.alpha * (reward + self.gamma * self.q_table[next_state][next_action] - self....
```

[task3/deep_agent.py] block 1

 # TODO: implement soft updates of the target network parameters

```
        for target_param, param in zip(self.target_network.parameters(), self.q_network.parameters()):
            target_param.data.copy_(self.tau * param.data + (1 - self.tau) * target_param.data)
```

[task3/dqn.py] block 1

 # TODO: compute the DQN loss

```
        print(f"Batch: {batch}")
        states = batch["states"]
        actions = batch["actions"]
        rewards = batch["rewards"]
        next_states = batch["next_states"]
        dones = batch["dones"]

        q_values = self.q_network(states).gather(1, actions.unsqueeze(1)).squeeze(1)
        with torch.no_grad():
            next_q_values = self.target_network(next_states).max(1)[0]
            target_q_values = rewards + (1 - dones) * self.gamma * next_q_values
        loss = F.mse_loss(q_values, target_q_values)
```

[task3/ppo.py] block 1

 # TODO: Compute the GAE advantage estimate for step t.

```
            delta = rewards[t] + self.agent.gamma * next_values * next_non_terminal - values[t]
            advantages[t] = delta + self.agent.gamma * self.agent.gae_lambda * next_non_terminal * last_gae
            last_gae = advantages[t]
```

[task3/ppo.py] block 2

 # TODO: compute the PPO clipped objective loss

```
                policy_loss = -torch.min(ratio * mb_advantages, torch.clamp(ratio, 1 - self.agent.clip_coef, 1 + self.ag...
```